

Elementary Sorting Algorithms

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

ADOPT

GHOST

FILMS

A D O P T

G H O S T

F I L M S

A D O P T

G H O S T

F I L M S

A	D	O	P	T
----------	----------	----------	----------	----------

G	H	O	S	T
----------	----------	----------	----------	----------

F	I	L	M	S
----------	----------	----------	----------	----------

Discussion

- How did you know?
- How certain are you?

Sorting Problem

- One of the foundational problems in computer science
- You are given a sequence (or list) of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
- **Reorder** the n numbers $\langle a_1', a_2', \dots, a_n' \rangle$ (i.e., permutation) such that they are arranged in a **specific order**, *usually* in **ascending** manner
- **Ascending Order Requirement:** $a_1' \leq a_2' \leq \dots \leq a_n'$

Sorted List of Numbers

1	2	3	4	5
----------	----------	----------	----------	----------

Not a Sorted List of Numbers

2	1	5	3	4
---	---	---	---	---

Notes

- Therefore, our goal is to rearrange the numbers
- Why?
- How?

Why?

- As we have observed in our previous lecture, doing a search on sorted data is generally easier and **faster** (recall: **binary search**; not BST)
- Also, it is easier for us to manipulate our data
- However, take note that sorting data also comes with a cost

How?

- Whenever you propose a solution, the steps must be **consistent**
- It must be **systematic** and has a logical flow
- Think of these steps as a series or **set of instructions**
- In short, you are defining an **algorithm**

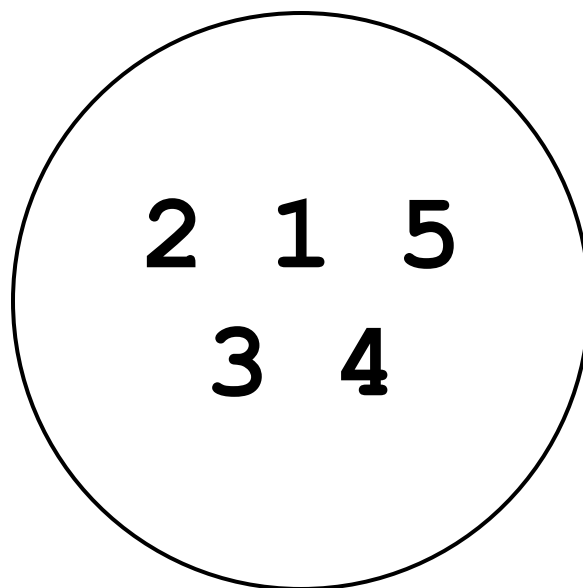
Naïve Approach /1

What would be a good way to solve the sorting problem?

Naïve Approach /2

- Perhaps we put all the numbers in a box
- Prepare the placeholder that will hold the individual numbers
- Get one number at a time then repeat until you run out of numbers
- Finally, we check if the requirement is *satisfied*

--	--	--	--	--



5 choices

4 choices

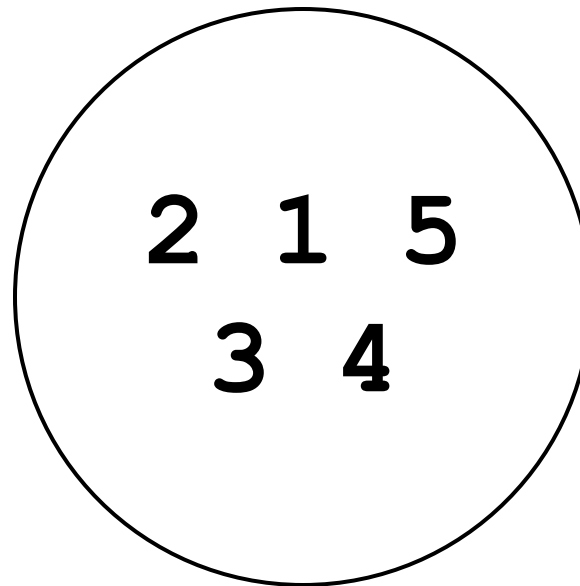
3 choices

2 choices

1 choice

--	--	--	--	--

therefore, there are $5!$ possible orders!



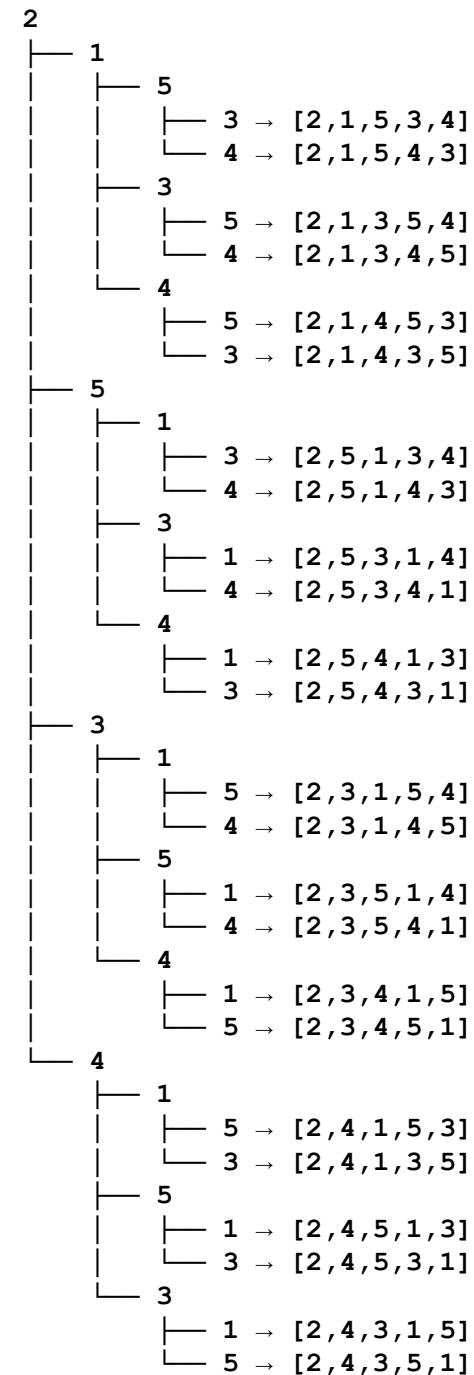
So, $O(n!)$

but for each, we do a
check to see if it is
sorted, so really
 $O(n \cdot n!)$

Permutation Example

This is only a portion of the solution

Thoughts?



Notes

- We tried all possible arrangements (i.e., permutation)
- Unfortunately, only one arrangement is correct*

Discussion

What do you think is the running time of this solution?

Elementary Sorting Algorithms /1

- Basic sorting methods in computer science
- Logic is straightforward and **easy** to implement

Elementary Sorting Algorithms /2

- Selection Sort
- Insertion Sort
- Bubble Sort

Selection Sort /1

- You will have a **logical partition**: the sorted and the unsorted regions
- We work from left to right by trying to fill in the correct element for the current location
- Sorted region grows from the left

Selection Sort /2

- Look for the **minimum element** from the unsorted region
- **Swap** the elements at the current location you are filling up with the minimum element found; consequently, the sorted region grows
- Repeat until there is only one item left in the unsorted region

5	3	8	4	2
---	---	---	---	---

5	3	8	4	2
---	---	---	---	---

i

min

5	3	8	4	2
---	---	---	---	---

i

j

min

found new min

5	3	8	4	2
---	---	---	---	---

i

j

min

5	3	8	4	2
---	---	---	---	---

i

j

min

5	3	8	4	2
---	---	---	---	---

i

j

min

5	3	8	4	2
---	---	---	---	---

i

j

min

found new min

5	3	8	4	2
---	---	---	---	---

i

j

min

5	3	8	4	2
---	---	---	---	---

i

min

swap

2	3	8	4	5
----------	---	---	---	----------

i

min

2	3	8	4	5
---	---	---	---	---

i

2	3	8	4	5
---	---	---	---	---

i

2	3	8	4	5
---	---	---	---	---

i

2	3	8	4	5
---	---	---	---	---

i

min

2	3	8	4	5
---	---	---	---	---

i

j

min

2	3	8	4	5
---	---	---	---	---

i

j

min

2	3	8	4	5
---	---	---	---	---

i

j

min

2	3	8	4	5
---	---	---	---	---

i

min

2	3	8	4	5
---	---	---	---	---

i

2	3	8	4	5
---	---	---	---	---

i

2	3	8	4	5
---	---	---	---	---

i

2	3	8	4	5
---	---	---	---	---

i

min

2	3	8	4	5
---	---	---	---	---

i

j

min

found new min

2	3	8	4	5
---	---	---	---	---

i

j

min

2	3	8	4	5
---	---	---	---	---

i

j

min

2	3	8	4	5
---	---	---	---	---

i

min

swap

2	3	4	8	5
---	---	---	---	---

i

min

2	3	4	8	5
---	---	---	---	---

i

2	3	4	8	5
---	---	---	---	---

i

2	3	4	8	5
---	---	---	---	---

i

2	3	4	8	5
---	---	---	---	---

i

min

2	3	4	8	5
---	---	---	---	---

i

j

min

found new min

2	3	4	8	5
---	---	---	---	---

i

j

min

2	3	4	8	5
---	---	---	---	---

i

min

swap

2	3	4	5	8
---	---	---	---	---

i

min

2	3	4	5	8
---	---	---	---	---

i

2	3	4	5	8
---	---	---	---	---

i

2	3	4	5	8
---	---	---	---	---

2	3	4	5	8
---	---	---	---	---

Discussion

- What do you think is the worst-case running time?
- What does the worst-case scenario look like?

```
void selection_sort(int *arr, int size) {  
    for(int i = 0; i < size-1; i++) {  
        int min_idx = i;  
  
        for(int j = i+1; j < size; j++) {  
            if( arr[j] < arr[min_idx] ) {  
                min_idx = j;  
            }  
        }  
  
        if( i != min_idx )  
            swap( &arr[i], &arr[min_idx] );  
    }  
}
```


Insertion Sort /1

- You will have a **logical partition**: the sorted and the unsorted regions
- We assume that the first element is already sorted
- Sorted region grows from the left

Insertion Sort /2

- For each of the unsorted elements, we find its **correct location** in the sorted area (right to left), thereafter, inserting it there
- Inserting involves allocating an empty slot, so **shifting** will be involved
- Repeat until no more elements in the unsorted region

5	3	8	4	2
---	---	---	---	---

5	3	8	4	2
---	---	---	---	---

5	3	8	4	2
---	---	---	---	---

i

5	3	8	4	2
---	---	---	---	---

i

tmp = 3

5	3	8	4	2
---	---	---	---	---

j

i

tmp = 3

shift

5	5	8	4	2
---	---	---	---	---

j

i

tmp = 3

found

5	5	8	4	2
---	---	---	---	---

j

i

tmp = 3

insert

3	5	8	4	2
---	---	---	---	---

j

i

tmp = 3

3	5	8	4	2
---	---	---	---	---

i

3	5	8	4	2
---	---	---	---	---

i

3	5	8	4	2
---	---	---	---	---

i

3	5	8	4	2
---	---	---	---	---

i

3	5	8	4	2
---	---	---	---	---

i

tmp = 8

found

3	5	8	4	2
---	---	---	---	---

j

i

tmp = 8

insert

3	5	8	4	2
---	---	---	---	---

j

i

tmp = 8

3	5	8	4	2
---	---	---	---	---

j

i

3	5	8	4	2
---	---	---	---	---

j

i

3	5	8	4	2
---	---	---	---	---

i

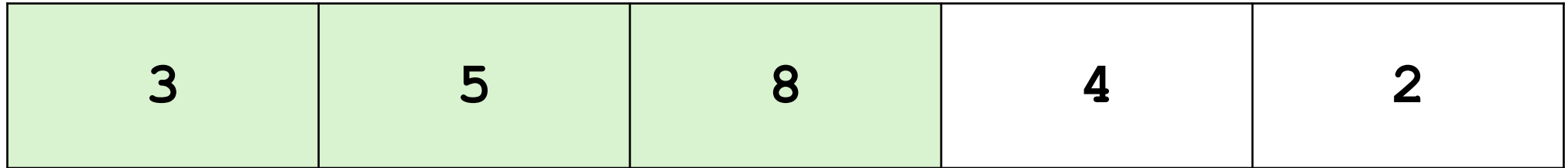
3	5	8	4	2
---	---	---	---	---

i

3	5	8	4	2
---	---	---	---	---

i

tmp = 4

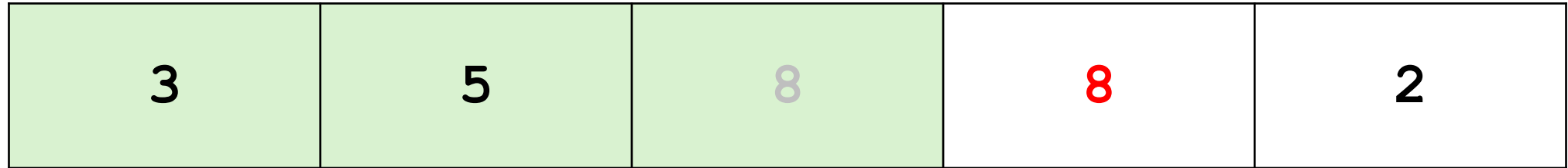


j

i

tmp = 4

shift



j

i

tmp = 4

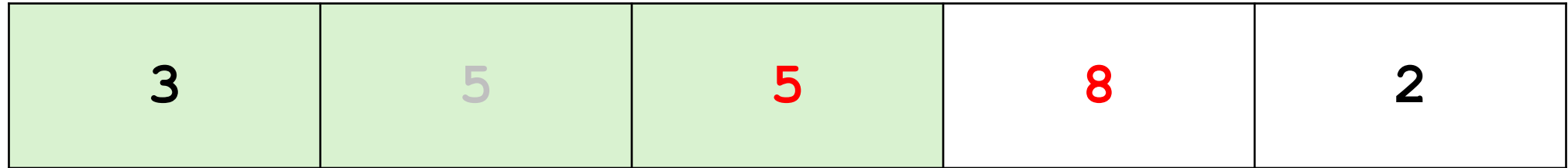
3	5	8	8	2
---	---	---	---	---

j

i

tmp = 4

shift



j

i

tmp = 4

found

3	5	5	8	2
---	---	---	---	---

j

i

tmp = 4

insert

3	4	5	8	2
---	---	---	---	---

j

i

tmp = 4

3	4	5	8	2
---	---	---	---	---

j

i

3	4	5	8	2
---	---	---	---	---

i

3	4	5	8	2
---	---	---	---	---

i

3	4	5	8	2
---	---	---	---	---

i

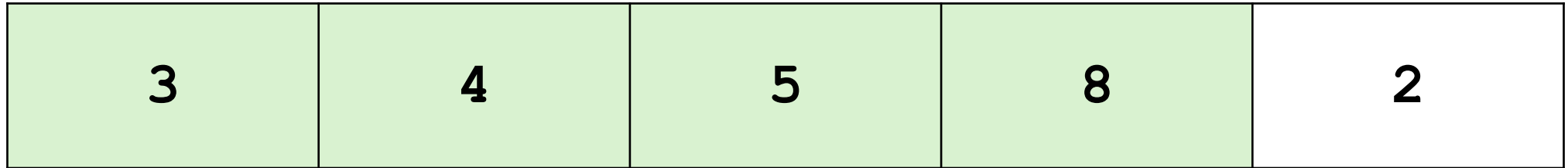
3	4	5	8	2
---	---	---	---	---

i

3	4	5	8	2
---	---	---	---	---

i

tmp = 2

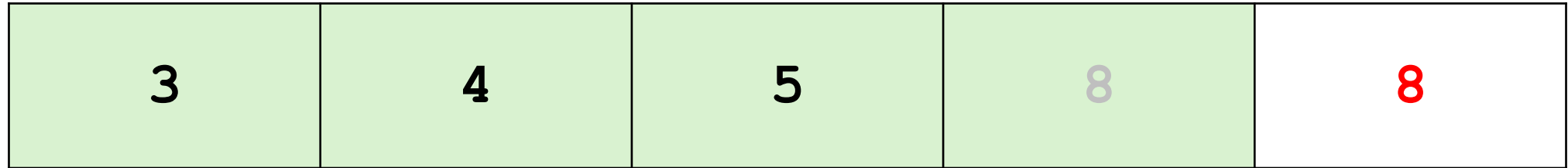


j

i

tmp = 2

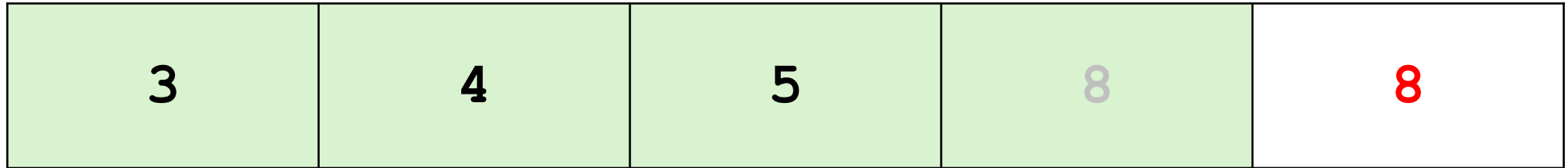
shift



j

i

tmp = 2



j

i

tmp = 2

shift



j

i

tmp = 2

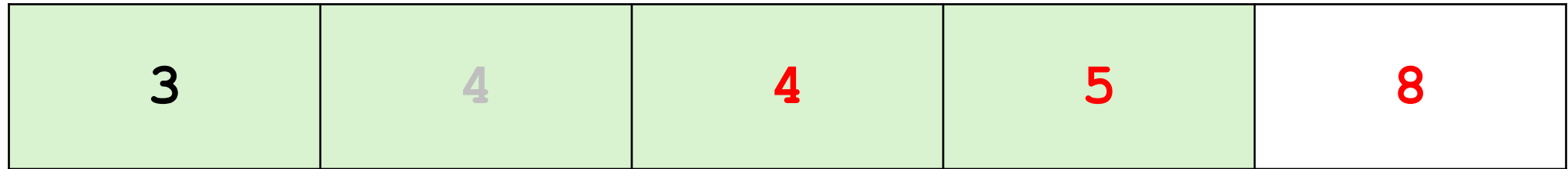


j

i

tmp = 2

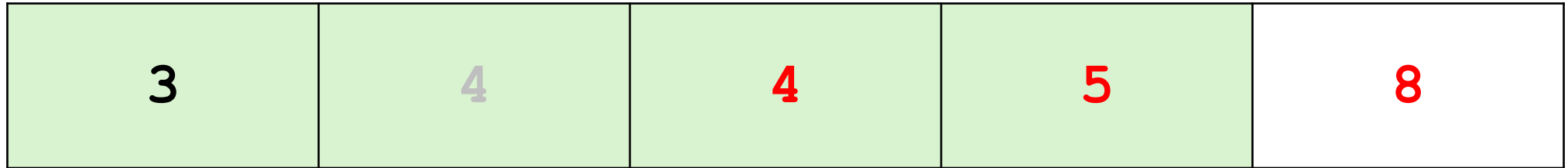
shift



j

i

tmp = 2

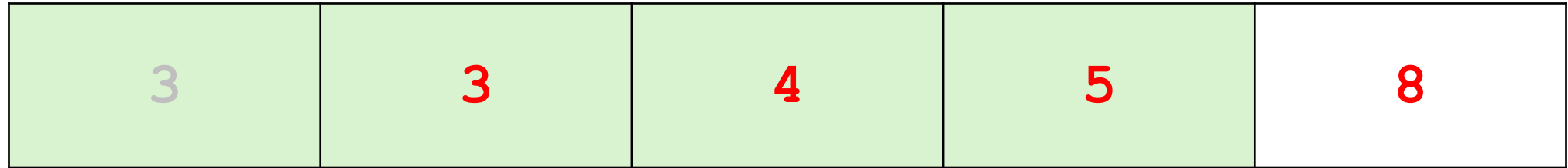


j

i

tmp = 2

shift



j

i

tmp = 2

found

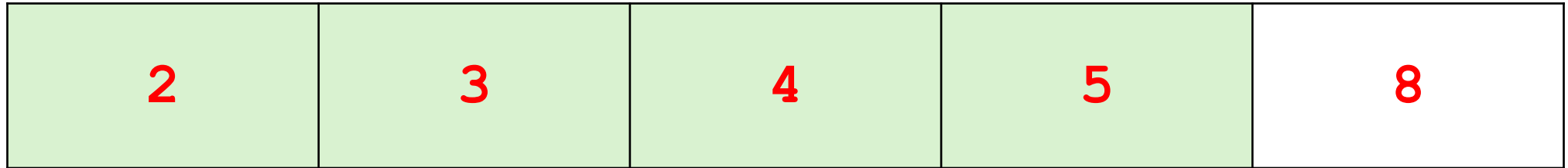
3	3	4	5	8
---	---	---	---	---

j

i

tmp = 2

insert



tmp = 2

2	3	4	5	8
---	---	---	---	---

j

i

2	3	4	5	8
---	---	---	---	---

i

2	3	4	5	8
---	---	---	---	---

i

2	3	4	5	8
---	---	---	---	---

i

2	3	4	5	8
---	---	---	---	---

Discussion /1

- Insertion sort shifts or moves elements only when necessary
- If elements are already sorted, no substantial moves are performed
- However, notice that once an element becomes part of the sorted region, its current location is not guaranteed to be its final one

Discussion /2

- What do you think is the worst-case running time?
- What does the worst-case scenario look like?

```
void insertion_sort(int *arr, int size) {  
    for(int i = 1; i < size; i++) {  
        int tmp = arr[i];  
        int j;  
  
        for(j = i-1; j >= 0 && tmp < arr[j]; j--) {  
            arr[j+1] = arr[j];  
        }  
  
        arr[j+1] = tmp;  
    }  
}
```

Your Turn!

- What do you think is the best-case running time?
- What does the best-case scenario look like?

Bubble Sort /1

- You will have a **logical partition**: the sorted and the unsorted regions
- For every ***round***, the general strategy is to “bubble” or push down the **heaviest element** of the unsorted region toward the end of the unsorted region
- Repeat for ***several rounds*** until one item is left in the unsorted region
- Sorted region grows from the right (i.e., implied)

Bubble Sort /2

- For each element in the unsorted region, check if it is larger than the element to its right, if yes, **swap** them (i.e., push heavier to the right)
- Repeat the process until you reach the second to the last element of the unsorted region
- As a result, the heaviest element of the unsorted region is pushed down to the end of the unsorted region

5	3	8	4	2
---	---	---	---	---

Round $i = 0$

5	3	8	4	2
---	---	---	---	---

j

5	3	8	4	2
---	---	---	---	---

j

$j+1$

swap

3	5	8	4	2
---	---	---	---	---

j

j+1

3	5	8	4	2
---	---	---	---	---

j

3	5	8	4	2
---	---	---	---	---

j

j+1

3	5	8	4	2
---	---	---	---	---

j

3	5	8	4	2
---	---	---	---	---

j

j+1

swap

3	5	4	8	2
---	---	---	---	---

j

j+1

3	5	4	8	2
---	---	---	---	---

j

3	5	4	8	2
---	---	---	---	---

j

j+1

swap

3	5	4	2	8
---	---	---	---	---

j

j+1

.

3	5	4	2	8
---	---	---	---	---

3	5	4	2	8
---	---	---	---	---

3	5	4	2	8
---	---	---	---	---

Round i = 1

3	5	4	2	8
---	---	---	---	---

3	5	4	2	8
---	---	---	---	---

j

3	5	4	2	8
---	---	---	---	---

j

j+1

3	5	4	2	8
---	---	---	---	---

j

3	5	4	2	8
---	---	---	---	---

j

j+1

swap

3	4	5	2	8
---	---	---	---	---

j

j+1

3	4	5	2	8
---	---	---	---	---

j

3	4	5	2	8
---	---	---	---	---

j

j+1

swap

3	4	2	5	8
---	---	---	---	---

j

j+1

3	4	2	5	8
---	---	---	---	---

3	4	2	5	8
---	---	---	---	---

3	4	2	5	8
---	---	---	---	---

Round i = 2

3	4	2	5	8
---	---	---	---	---

3	4	2	5	8
---	---	---	---	---

j

3	4	2	5	8
---	---	---	---	---

j

j+1

3	4	2	5	8
---	---	---	---	---

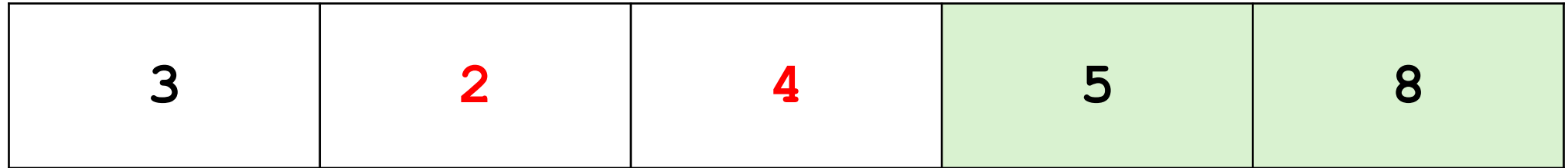
j

3	4	2	5	8
---	---	---	---	---

j

j+1

swap



j

j+1

3	2	4	5	8
---	---	---	---	---

3	2	4	5	8
---	---	---	---	---

3	2	4	5	8
---	---	---	---	---

Round i = 3

3	2	4	5	8
---	---	---	---	---

3	2	4	5	8
---	---	---	---	---

j

3	2	4	5	8
---	---	---	---	---

j

j+1

swap

2	3	4	5	8
---	---	---	---	---

j

j+1

2	3	4	5	8
---	---	---	---	---

2	3	4	5	8
---	---	---	---	---

2	3	4	5	8
---	---	---	---	---

2	3	4	5	8
---	---	---	---	---

Discussion /1

- What do you think is the worst-case running time?
- What does the worst-case scenario look like?

```
void bubble_sort(int *arr, int size) {  
    for(int i = 0; i < size-1; i++) {  
        for(int j = 0; j < size-1-i; j++) {  
            if( arr[j] > arr[j+1] ) {  
                swap( &arr[j], &arr[j+1] );  
            }  
        }  
    }  
}
```

Discussion /2

- Do you think we can improve the current algorithm for bubble sort?
- Hint: What if the elements are already sorted to begin with?
- How will the algorithm proceed?

Your Turn!

For each of the three elementary sorting algorithms, perform it by hand and record the state of the following array after each iteration of the outer loop.

7, 4, 5, 2, 9, 1, 6

Challenge

Complete the function that accepts an array of distinct strings and its size. It sorts the strings in alphabetical order. You may assume that all strings consist only of lowercase English letters.

```
void sort(char **words, int size) {
```


Final Notes /1

- We can see that a problem can be solved in many ways
- Compared to our naïve approach to solving the sorting problem, these basic algorithms are *relatively faster*

Final Notes /2

- We explored three basic sorting algorithms and analyzed their worst-case running time
- We can now compare the worst-case running time of the sorting algorithms we have learned so far

Final Notes /3

- In our discussion, we assumed that our data are distinct
- What happens when there are duplicates in our list?
- Look into the concept of **stable sorting algorithms**

3 4 4 2 $\Rightarrow$ 2 3 4 4
 A B A B

Check Your Understanding

Sorting Algorithm	Is Stable?	Worst-Case	Average Case	Best-Case
Selection Sort			$O(n^2)$	
Insertion Sort			$O(n^2)$	
Bubble Sort			$O(n^2)$	
Bubble Sort (Enhanced)			$O(n^2)$	

Questions?